

TAKSH PATEL

JavaScript Full Stack Developer

takshpatel.vercel.app | github.com/TakshPatel02 | linkedin.com/in/taksh-patel20 | takshpatel022@gmail.com

Full stack developer in the React/Node ecosystem, with a habit of pulling reusable pieces out of whatever I'm shipping and publishing them separately — a 68-component animation library, an Express auth CLI, an icon-motion package. More interested in the shape of a component or CLI's API than in getting a feature to just work.

TECHNICAL SKILLS

Languages JavaScript, TypeScript

Frontend React, HTML, CSS, Tailwind CSS v4, GSAP, Motion

Backend Node.js, Express.js, REST APIs, JWT (access/refresh flow), middleware design, cookies & sessions, Zod validation

Database MongoDB, Mongoose

Tooling Git, GitHub, VS Code, Postman, Vercel, npm

Currently learning Next.js, Docker, Redis, Prisma, System Design

SELECTED ENGINEERING PROJECTS

Developer Portfolio & Open-Source Portfolio Template *React → Next.js migration*

- Built a reusable portfolio platform rather than a one-off personal website, separating application logic from content through a configuration driven architecture for easy customization and long-term maintainability.
- Architected modular sections for projects, technical writing, experience, resume, and GitHub integration, enabling the portfolio to evolve as new content is added without structural changes.
- Emphasized **developer experience** with detailed documentation, reusable components, and deployment guidance, allowing other developers to adopt and personalize the template efficiently.
- Used the portfolio as a platform to showcase open-source work and technical writing, attracting **450+ visitors** with an average engagement time exceeding **2 minutes**.

ComponentLabs — published component library *npm · component-labs · 68 components*

- Published a 68-component React library with 3 page transitions and 3 theme transitions, all theme and page transitions are copy-paste (no install step) — chosen deliberately over a package-install model because the target user is copying one transition into an existing codebase, not adopting a design system.
- Enforced a one-primary-motion-per-component rule and CSS transforms over JS-driven animation across all 68 components, so the library reads as a coherent set of decisions, not a grab-bag of effects.

AI Code Reviewer — full-stack app, two repos *React 19/Vite frontend + Node/Express/Gemini backend*

- Split frontend and backend into separately deployed repos, with a hard 5-review cap per user baked into the API so history stays a bounded, predictable read instead of an unbounded collection.
- Auth flow issues a short-lived access token plus an httpOnly, secure, sameSite=strict refresh cookie, so the refresh token is never reachable from JS on the frontend even though the access token is.

Bank Management System *Node.js · Express · MongoDB*

- Modeled account balances as a ledger of transactions rather than a single mutable balance field, so every credit/debit is auditable and the current balance is a derived value, not a source of truth that can drift.
- Role-based access control and Zod-validated request bodies at every write endpoint, so a teller-level role and an admin-level role hit meaningfully different checks rather than a single is-admin flag.

OPEN SOURCE

Three published npm packages, maintained past the first release — versioning is intentional because a bad release breaks someone else's install, and breaking changes ship with a migration note instead of a silent rewrite.

component-labs — 68-component React animation library with 3 page and 3 theme transitions, all copy-paste, no install step required to use a single component.

iconflow — animated-icon companion package, kept independently versioned and installable from component-labs rather than bundled in.

create-express-authkit — CLI that scaffolds a production-shaped Express auth backend (JWT, Zod validation, rate limiting, password reset) in one command, with an interactive JS/TS prompt; JS/TS both are the maintained path.

TECHNICAL WRITING

Writes on Hashnode (takshpatel02.hashnode.dev) on Node.js security and system design — using writing as a way to pressure-test whether an explanation actually holds up outside a codebase, not as a content-marketing exercise.

EDUCATION

Bachelor of Engineering, Information Technology

Sal College of Engineering (Gujarat Technological University) · CGPA 9.28 · Expected 2027